Class 04 - Functions

Scope, enlace de argumentos y por que una funcion puede usar una variable global sin que lo notes

Ruta actual: week_01_logic -> class_04_functions

1. El caso que acabas de descubrir

Cambiaste los parametros de la funcion de `m0` y `mf` a nombres mas explicitos como `initial_mass` y `final_mass`. Sin embargo, dentro del cuerpo de la funcion seguian apareciendo `m0` y `mf`.

La llamada si estaba pasando correctamente los argumentos. El problema era que el cuerpo de la funcion no estaba usando esos parametros.

```
def calculate_rocket_delta_v(Isp, initial_mass, final_mass, g0=9.80665): mass_ratio = m0 / mf # <- usa nombres externos delta_v = Isp * g0 * np.log(mass_ratio) return delta_v
```

Si fuera de la funcion existen:

```
m0 = 500000 mf = 250000
```

Python puede encontrarlas y usarlas. Por eso parecia que la funcion ignoraba los valores nuevos que pasabas en cada llamada.

2. Que ocurre al definir una funcion

Cuando Python encuentra una sentencia `def`, no ejecuta todavia el calculo. Crea un objeto funcion y lo asocia a un nombre.

```
def calculate_rocket_delta_v(Isp, initial_mass, final_mass, g0=9.80665): ...
```

En ese momento la funcion queda preparada para ser llamada. El cuerpo se ejecutara cada vez que aparezca una llamada.

3. Que ocurre al llamar una funcion

Supongamos:

```
engine_isp = 250 wet_mass = 500000 dry_mass = 100000 dv = calculate_rocket_delta_v(engine_isp, wet_mass, dry_mass)
```

Python enlaza los argumentos posicionales con los parametros por orden:

Argumento exterior	Parametro local
engine_isp = 250	Isp = 250
wet_mass = 500000	initial_mass = 500000
dry_mass = 100000	final_mass = 100000
(no enviado)	g0 = 9.80665

La funcion obtiene asi un conjunto de nombres locales para esa llamada concreta.

4. Scope: donde existe cada nombre

Scope es el ambito donde un nombre puede encontrarse. Un parametro de una funcion es una variable local a esa llamada.

```
def example(initial_mass): ratio = initial_mass / 1000 return ratio
```

`initial_mass` y `ratio` son locales. Fuera de la funcion no necesitas que existan con esos nombres.

5. Como busca Python un nombre: LEGB

Cuando Python encuentra un nombre como `m0`, busca aproximadamente en este orden:

L	Local	variables de la funcion actual
E	Enclosing	funciones envolventes, si existen
G	Global	variables definidas en el modulo/archivo
B	Built-in	nombres integrados como len, print, etc.

En tu caso, al no existir `m0` ni `mf` como nombres locales, Python continuo buscando y encontro las variables globales definidas fuera.

6. Version incorrecta vs version correcta

Version peligrosa:

```
m0 = 500000 mf = 250000 def calculate_rocket_delta_v(Isp, initial_mass, final_mass): mass_ratio = m0 / mf return Isp * 9.80665 * np.log(mass_ratio)
```

Aunque llames a la funcion con masas diferentes, el calculo seguira usando las globales `m0` y `mf`.

Version correcta:

```
def calculate_rocket_delta_v(Isp, initial_mass, final_mass, g0=9.80665): mass_ratio = initial_mass / final_mass delta_v = Isp * g0 * np.log(mass_ratio) return delta_v
```

Ahora el resultado depende explicitamente de los valores que entran por los parametros.

7. Positional arguments y keyword arguments

Con argumentos posicionales, los nombres exteriores no tienen que coincidir con los parametros:

```
engine_isp = 250 wet_mass = 500000 dry_mass = 100000 dv = calculate_rocket_delta_v(engine_isp,
wet_mass, dry_mass)
```

Python enlaza por posicion. En cambio, con keyword arguments, el nombre a la izquierda del signo igual debe coincidir con el parametro:

```
dv = calculate_rocket_delta_v( Isp=engine_isp, initial_mass=wet_mass, final_mass=dry_mass )
```

La parte derecha puede llamarse como quieras; la izquierda identifica el parametro interno.

8. Que hace return

`return` finaliza inmediatamente la llamada y entrega un objeto al codigo que llamo a la funcion.

```
result = calculate_rocket_delta_v(...)
```

Si la funcion ejecuta `return None`, la llamada termina en ese punto y el codigo posterior del cuerpo no se ejecuta. Eso permite proteger calculos como una division por cero.

9. Por que preferimos parametros a globales ocultas

Una funcion cientifica es mas facil de entender, probar y reutilizar cuando sus dependencias son explicitas. Si necesita masa inicial y masa final, es mejor recibirlas como parametros que leerlas silenciosamente desde variables globales.

Regla practica: si un dato puede variar entre llamadas, normalmente debe entrar como parametro.

10. Prueba con el debugger

Pon un breakpoint dentro de la funcion y entra con F11. En la ventana de variables observa los Locals. Deberias ver `Isp`, `initial_mass`, `final_mass` y `g0`. Si el cuerpo usa `m0`, comprueba que no forma parte de esos parametros locales.

11. Checklist al renombrar parametros

1	Cambia el nombre en la cabecera def.
2	Cambia todas las referencias dentro del cuerpo.
3	Actualiza validaciones y mensajes.
4	Actualiza llamadas por keyword si existen.
5	Actualiza docstrings/documentacion.
6	Prueba con valores muy diferentes para detectar globals accidentales.

12. Modelo mental final

CALL -> bind arguments -> create local scope -> execute body -> name lookup if needed -> return -> back to caller

Learning Tracker

Concepto	Estado
Parametros locales	OK
Argumentos posicionales	OK
Keyword arguments	OK
Scope	OK
LEGB	Introducido
Globales accidentales	Detectadas
return / return None	OK
Debugger para inspeccionar Locals	Practicar